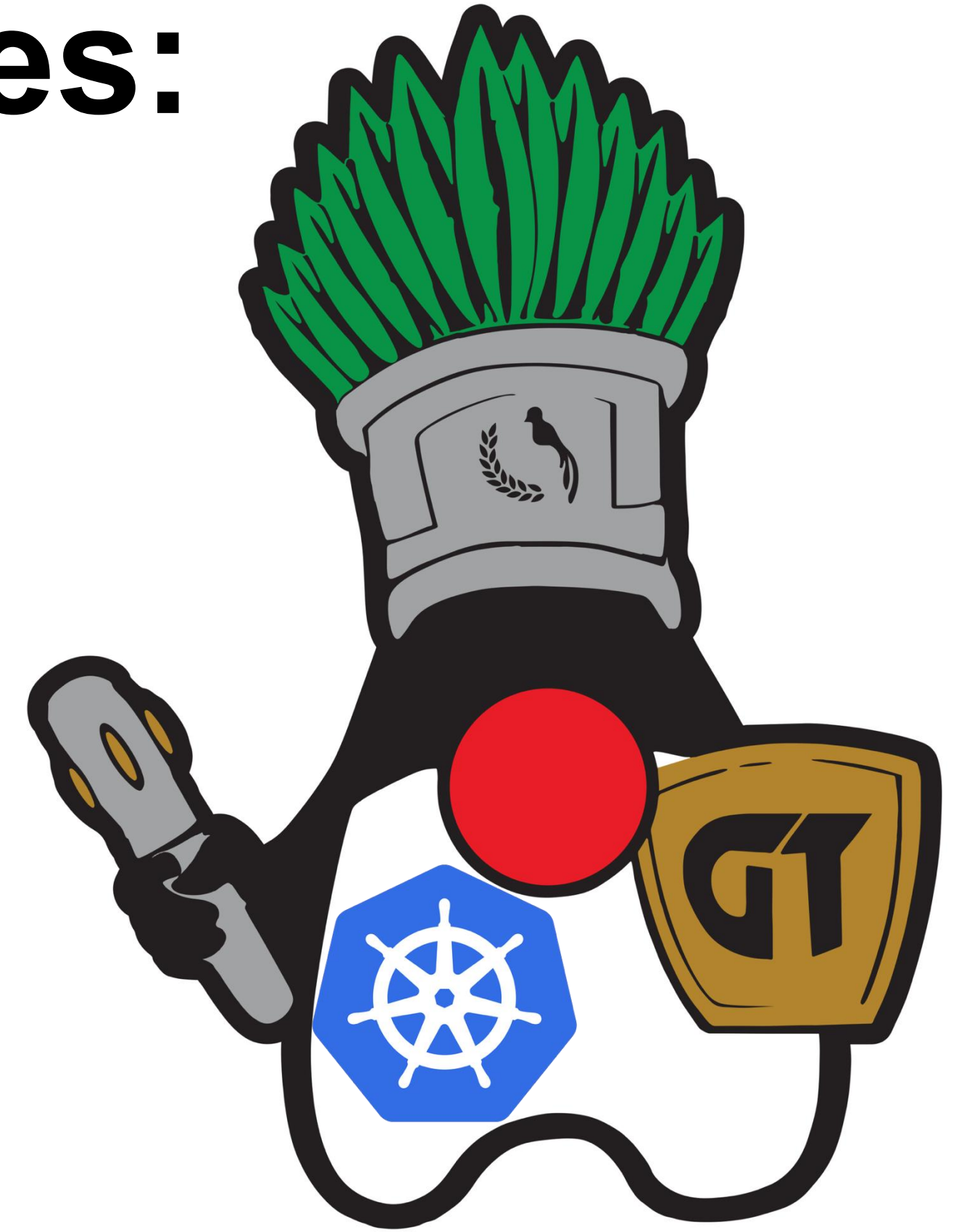


Del código a Kubernetes:

¿cómo request & limits
moldean el destino de
tus aplicaciones?



GUATE-JUG



K8S restaurant



HIGH PRIORITY



RESERVED

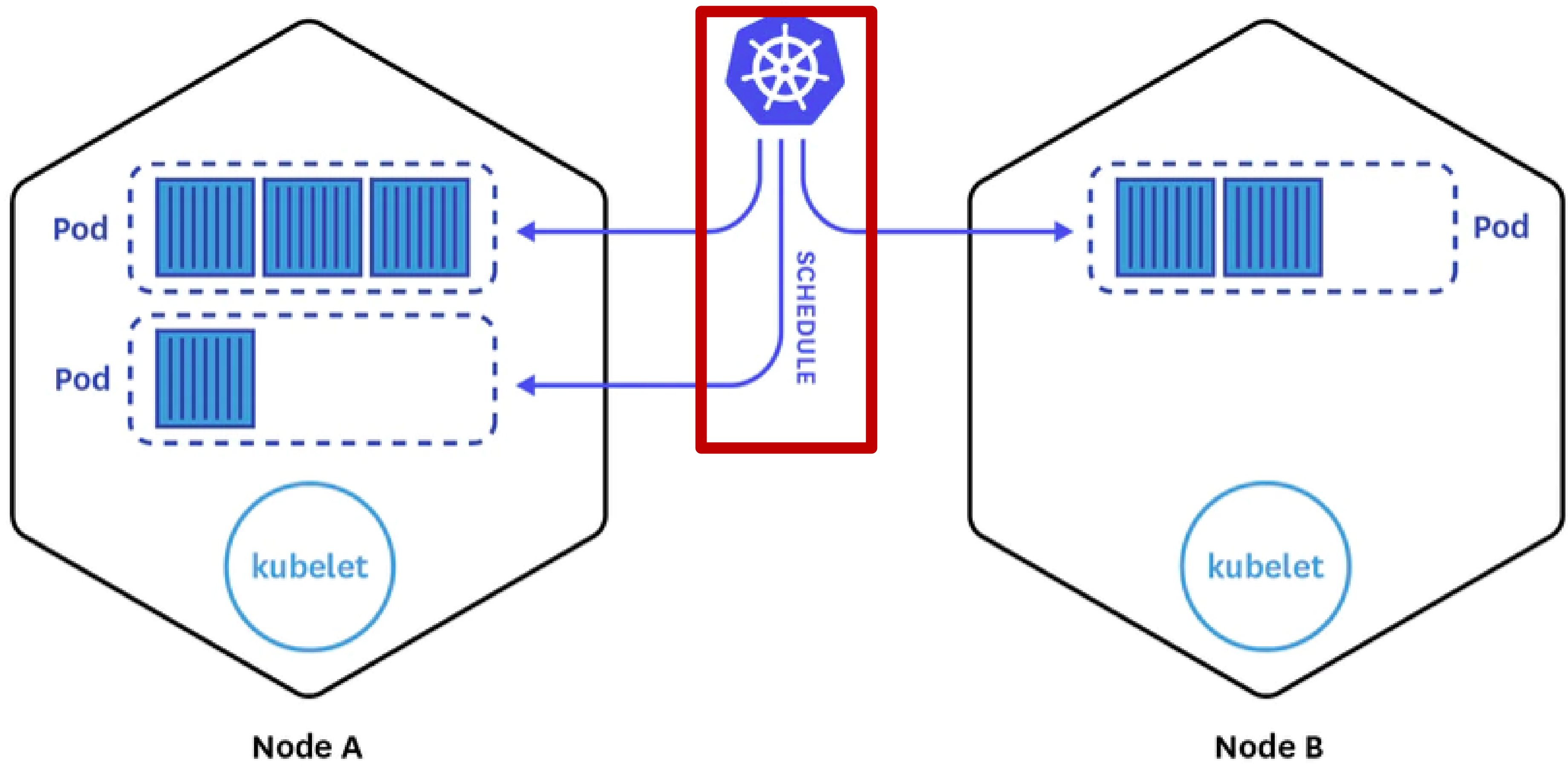


Adalberto García

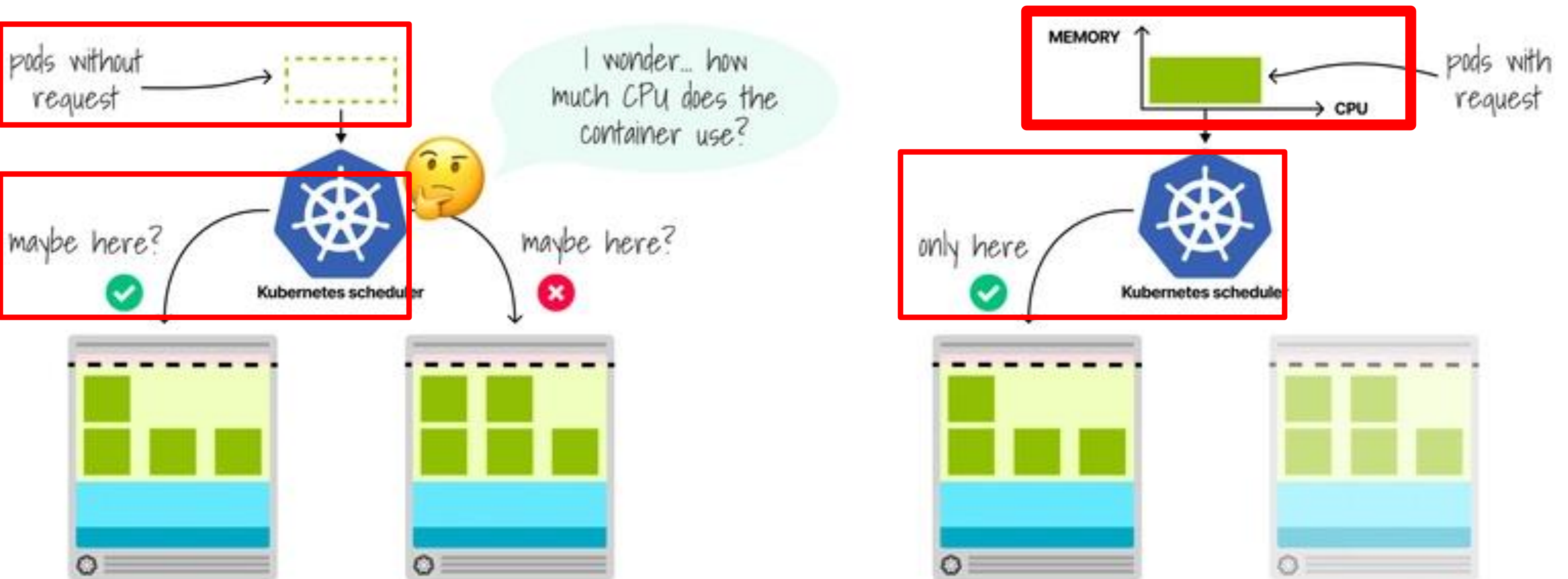
BYTE



www.linkedin.com/in/adalberto-garcia/



CLUSTER



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app1-nginx-deployment
  labels:
    app: app1-nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: app1-nginx
  template:
    metadata:
      labels:
        app: app1-nginx
    spec:
      containers:
        - name: app1-nginx
          image: stacksimplify/kube-nginxapp1:1.0.0
          imagePullPolicy: Always
          ports:
            - containerPort: 80
            # Requests & Limits for usermount-webapp Container
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "200m"
              memory: "256Mi"
```

request & limits

¿porqué algo tan importante es opcional y no obliga a definirlo de manera mandatoria?





Pod Priority

Quality of Service Classes (QoS)

Limits:

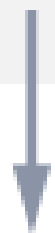
cpu: 1

memory: 500Mi

Requests:

cpu: 1

memory: 500Mi



Guaranteed

Limits:

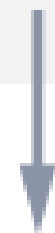
cpu: 500m

memory: 500Mi

Requests:

cpu: 300m

memory: 400Mi



Burstable

<No Limits or Requests>



BestEffort

OOM

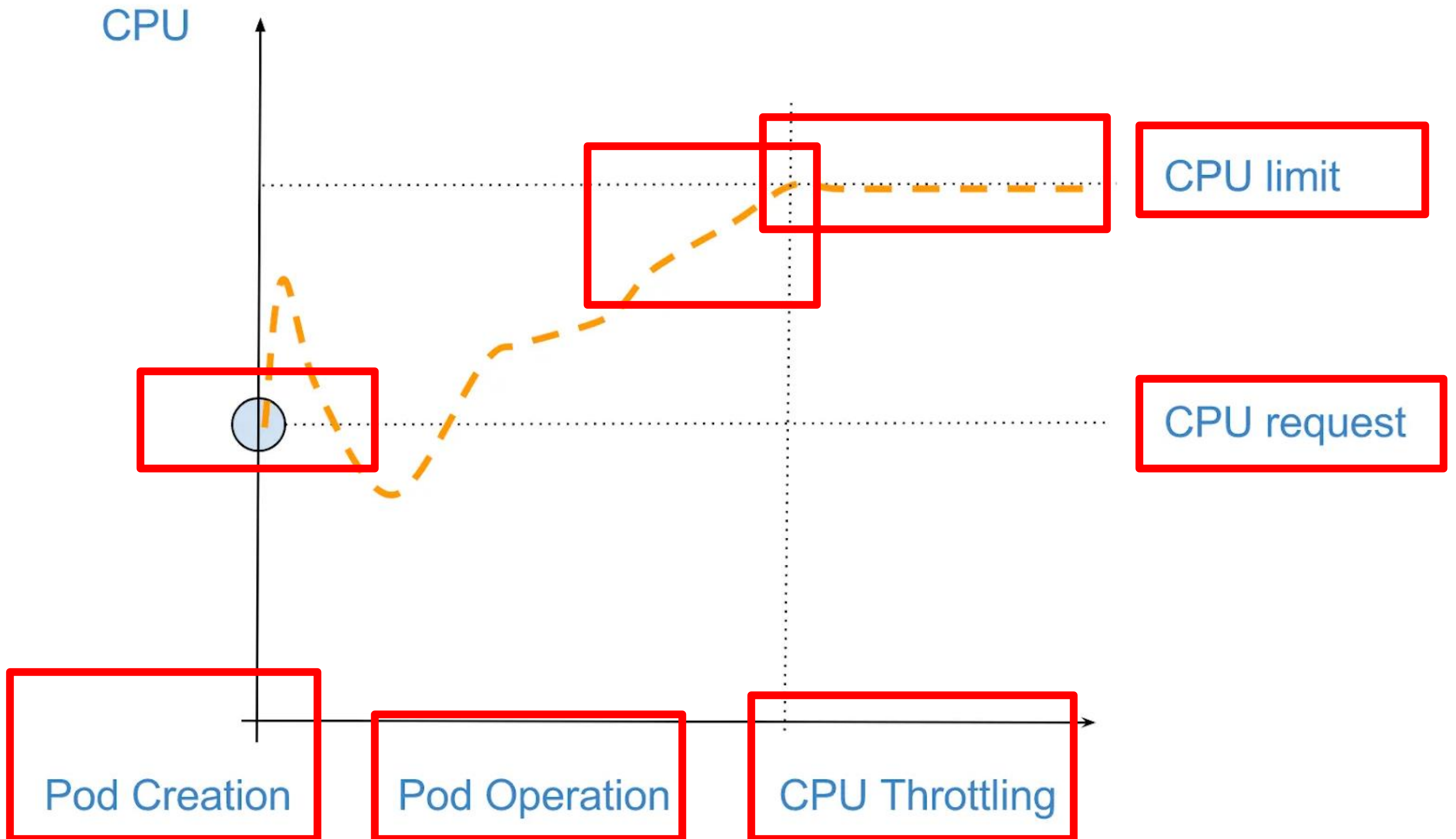


OOM

THROTTLING







¿Porqué es importante para Java?

Java = Arq. Multithread

Default stacks
(metrics, health, web-server)

8 vCPU

1

Using 8 GB of memory

4 vCPU

1 vCPU



Using 1.5 CPU

2

0

2GiB

4GiB

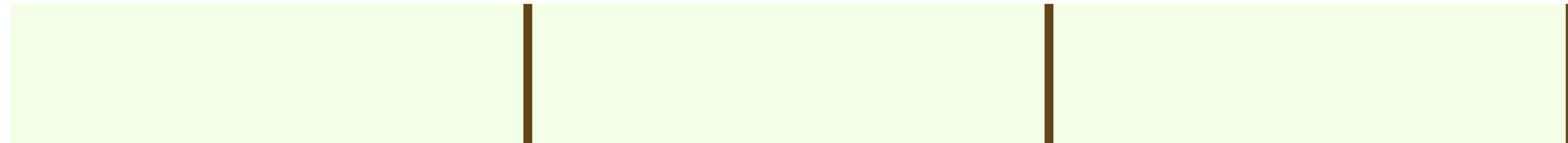
8GiB

16GiB

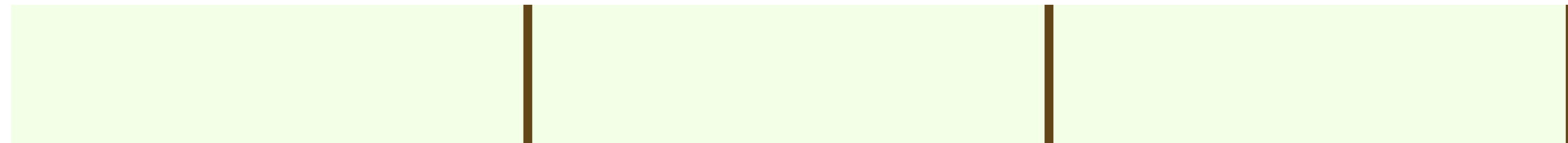
PERIOD 1

PERIOD 2

PERIOD 3



1 thread, 1 CPU



2 thread, 1 CPU

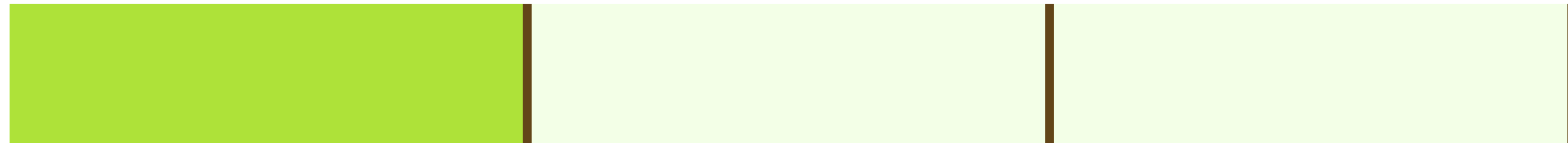


8 thread, 1 CPU

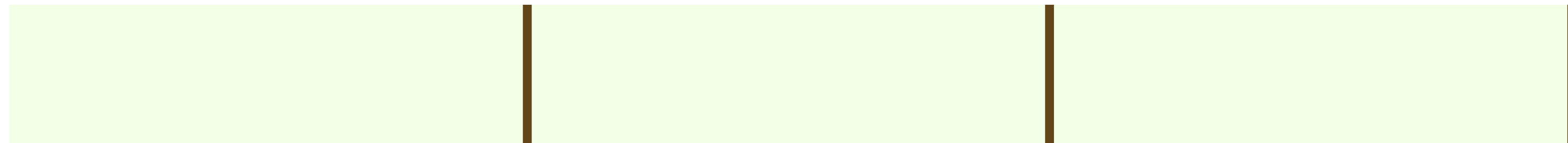
PERIOD 1

PERIOD 2

PERIOD 3



1 thread, 1 CPU



2 thread, 1 CPU

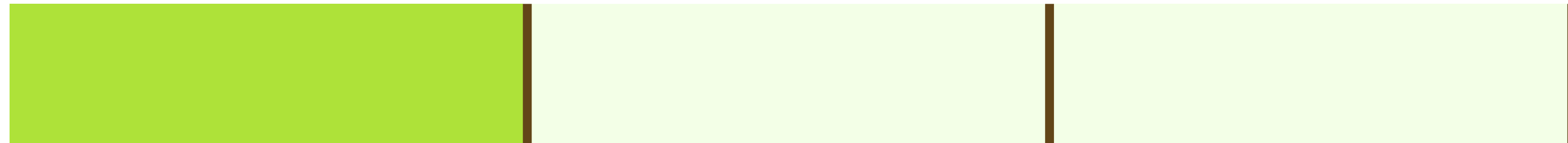


8 thread, 1 CPU

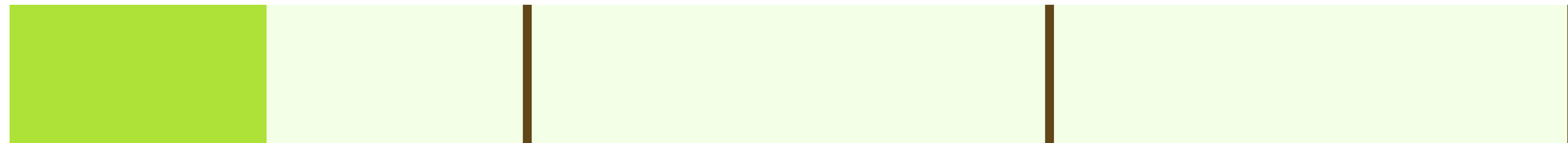
PERIOD 1

PERIOD 2

PERIOD 3



1 thread, 1 CPU



2 thread, 1 CPU

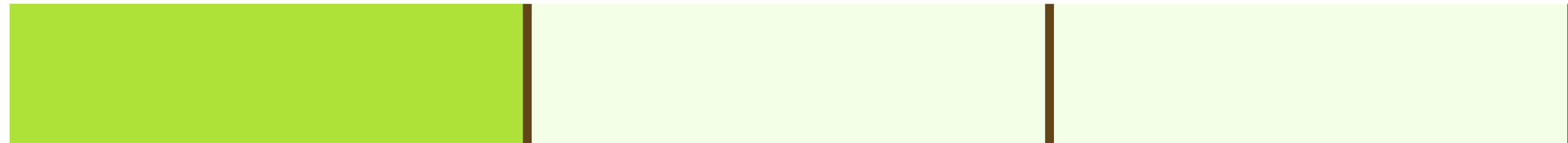


8 thread, 1 CPU

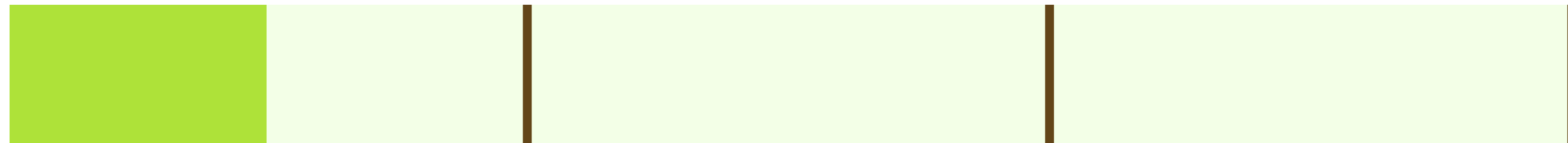
PERIOD 1

PERIOD 2

PERIOD 3



1 thread, 1 CPU



2 thread, 1 CPU



8 thread, 1 CPU

PERIOD 1

PERIOD 2

PERIOD 3



1 thread, 1 CPU



2 thread, 1 CPU



8 thread, 1 CPU

PERIOD 1

PERIOD 2

PERIOD 3



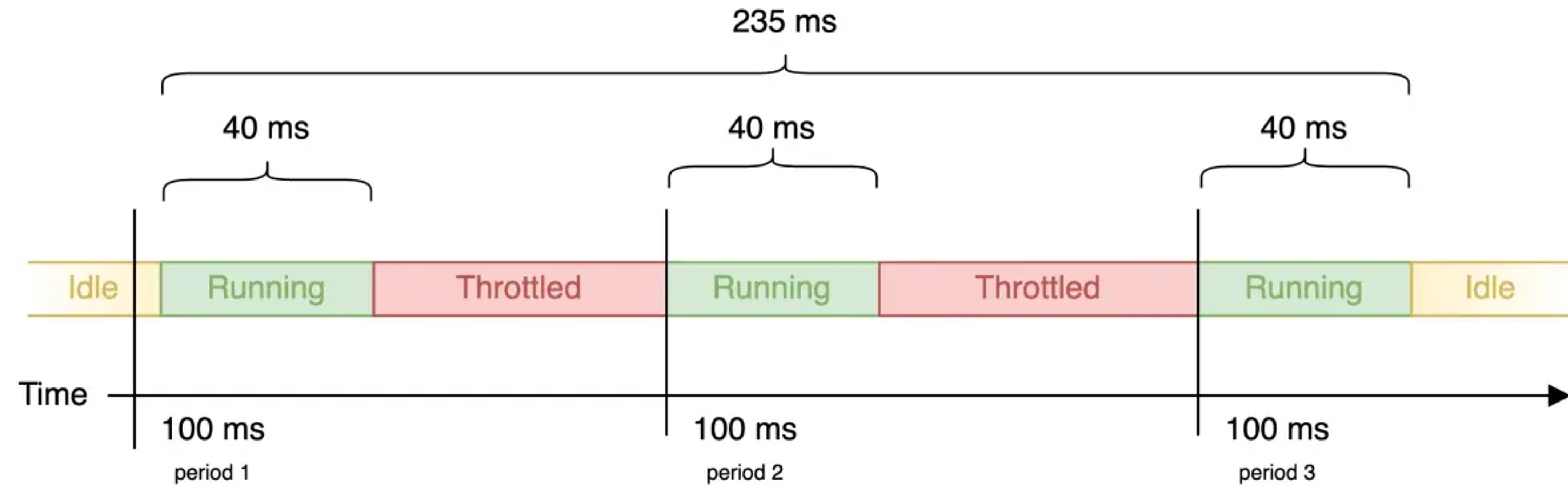
1 thread, 1 CPU

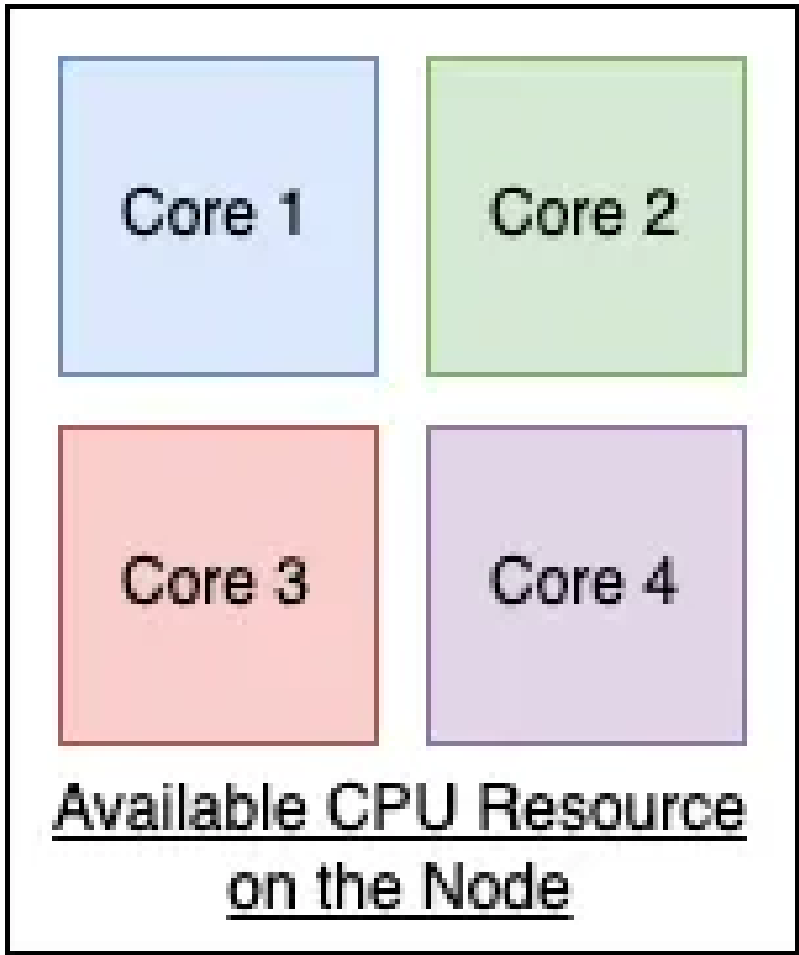


2 thread, 1 CPU



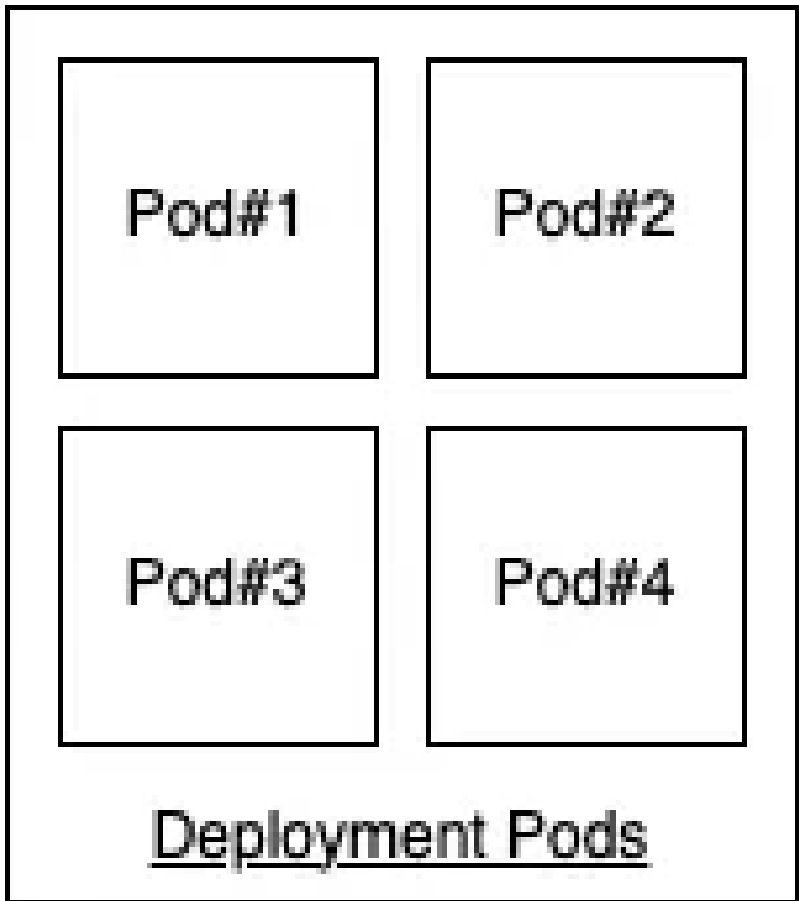
8 thread, 1 CPU





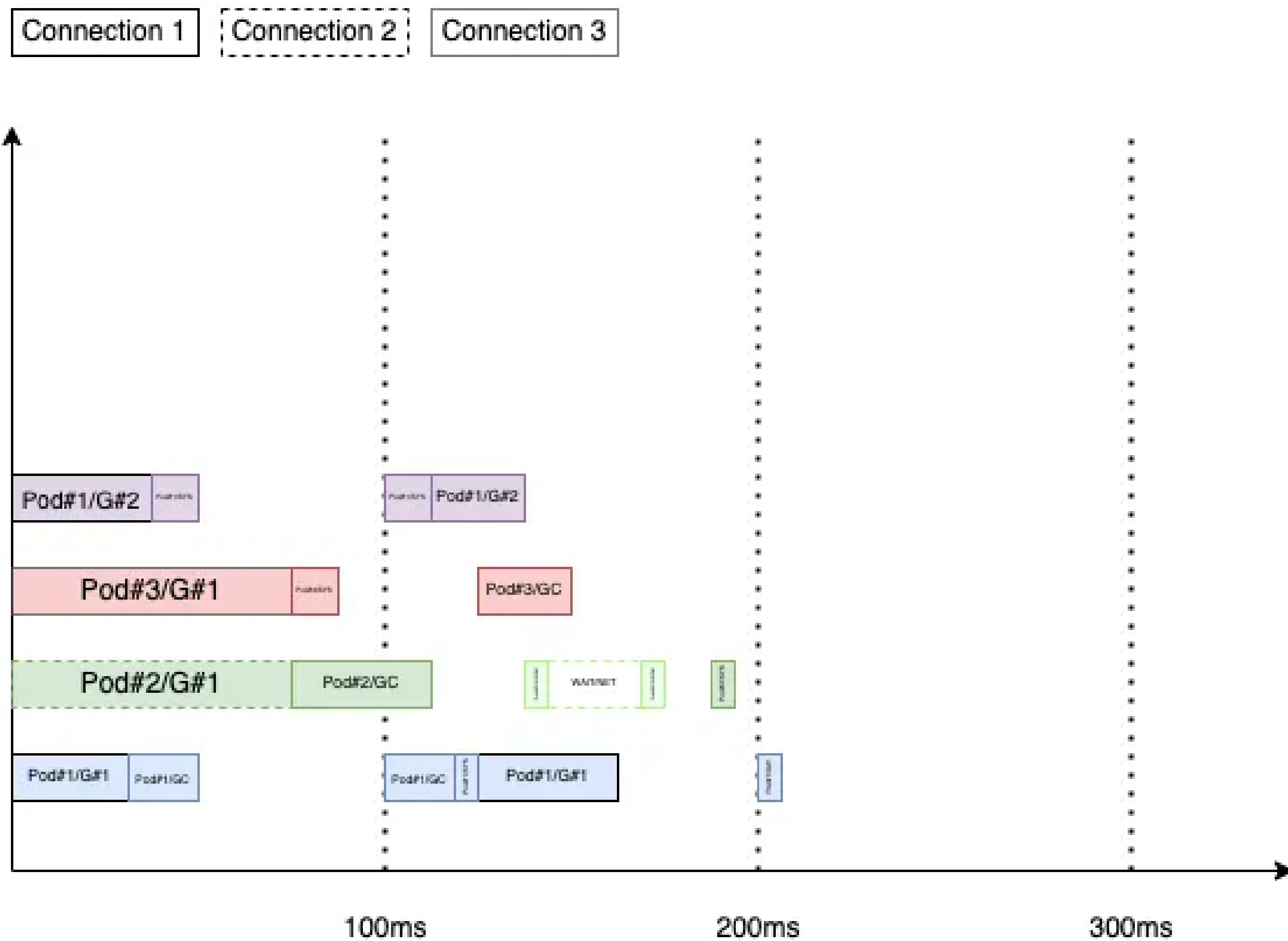
CPU Limit
1 core

Concurrency
4 Reqs



Required Exc Time
75ms

Keep Alive
On



Requests	Pod ID	Throttled	Response time
 Request #1 → Pod #1	Pod #1	87.5ms	206.5ms
 Request #2 → Pod #2	Pod #2	0ms	75ms
 Request #3 → Pod #3	Pod #3	0ms	75ms
 Request #4 → Pod #1	Pod #1	50ms	137ms

Core 1

Core 2

Core 3

Core 4

Available CPU Resource on the Node

CPU Limit

1 core

Concurrency

4 Reqs

Required Exc Time

75ms

Keep Alive

On

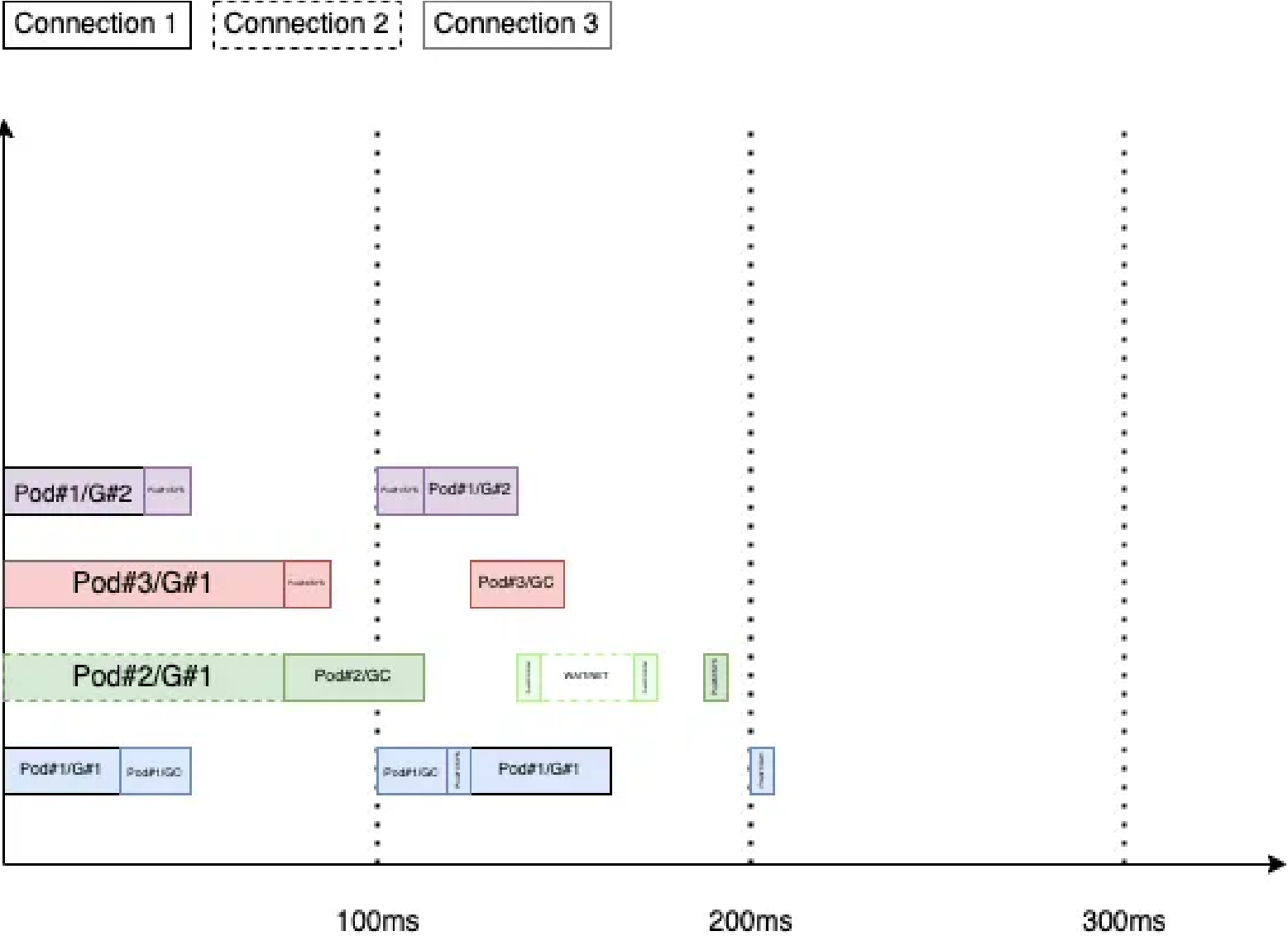
Pod#1

Pod#2

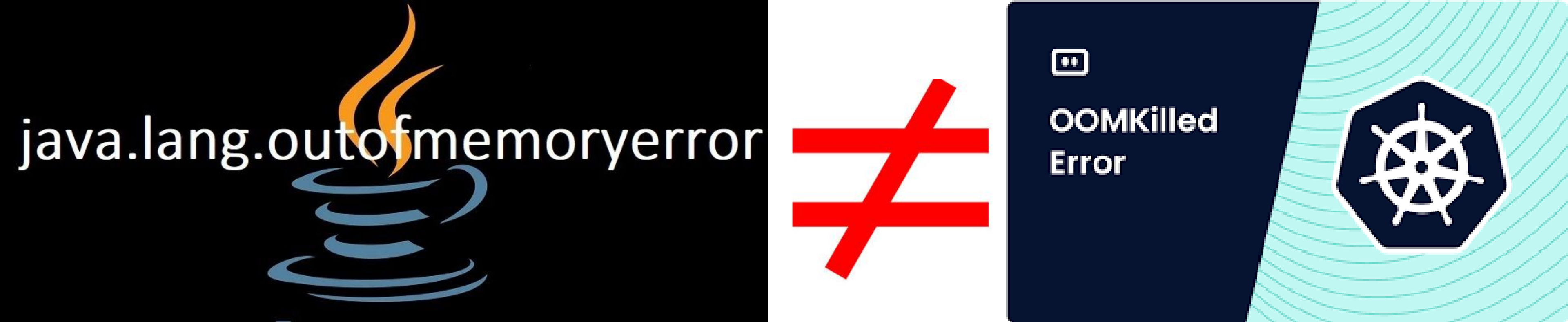
Pod#3

Pod#4

Deployment Pods

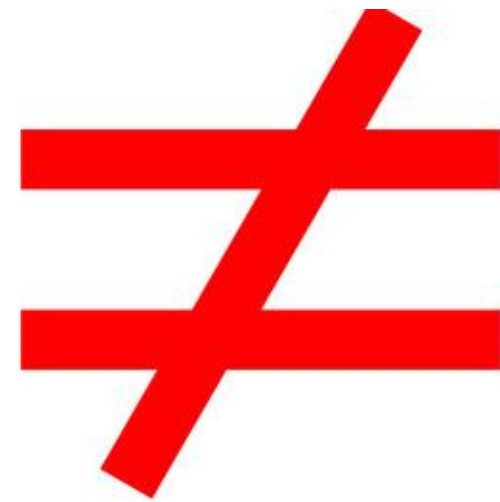


Requests	Pod ID	Throttled	Response time
Request #1	→ Pod #1	87.5ms	206.5ms
Request #2	→ Pod #2	0ms	75ms
Request #3	→ Pod #3	0ms	75ms
Request #4	→ Pod #1	50ms	137ms



JVM Reserva Heap Virtual (memory mapping)

Se asigna físico cuando realmente se usa



OME => Virtual (El Kernel no lo ve).

El Proceso sigue vivo

Kubernetes no mata el contenedor. No se produce OOMKill (JVM imprime StackTrace)

DEBATE

¿Deberíamos establecer límites de CPU o dejar que los contenedores tomen lo que necesitan?

Request & Limits

10 consejos prácticos
para no sufrir con tus
aplicaciones...

1

Siempre define CPU
request: garantiza
planificación adecuada

Usa Pod Priority: protege tus cargas críticas..

3

Evita CPU limit salvo que tengas razones técnicas claras para usarlos.

4

`kind: LimitRange`



`limits:`

`cpu:`
`"500m"`

`memory:`
`"1Gi"`

5

kind: ResourceQuota



hard:
cpu: "4"
memory:
"8Gi"

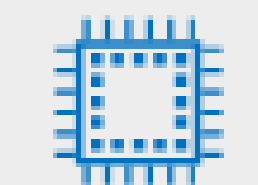
No es solo monitorear:
entiende el impacto del
throttling en tu aplicación
(Grafana / Prometheus)

Usa limits (al inicio)...

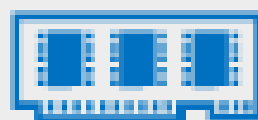
Hasta que sepas cuándo
no usarlos (Del límite fijo al
escalado inteligente).

8

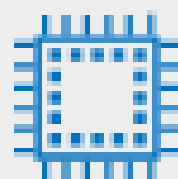
Horizontal Pod Autoscaler (HPA)



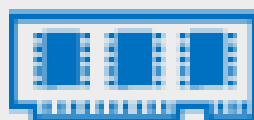
250m



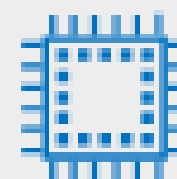
64Mi



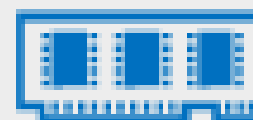
250m



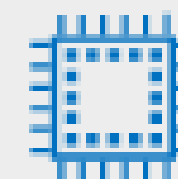
64Mi



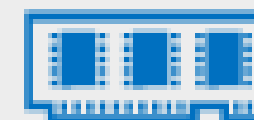
250m



64Mi

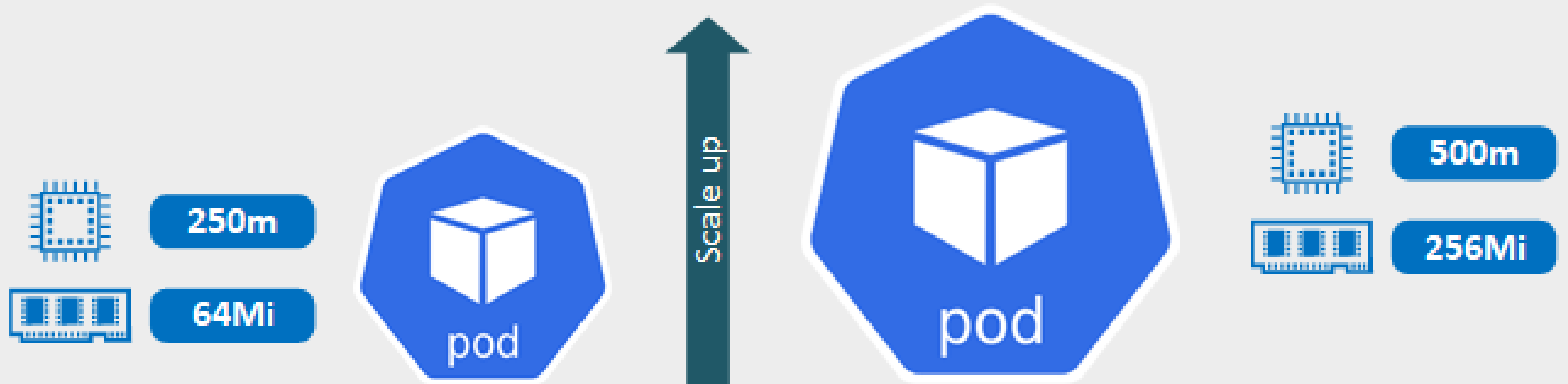


250m



64Mi

Vertical Pod Autoscaler (VPA)



10 Kubernetes Resource Recommender - krr

Scan result (99.96 points)

Number	Cluster	Namespace	Name	Pods	Type	Container	CPU Requests	CPU Limits	Memory Requests	Memory Limits
1.	gke_robusta-development_us-east5-a_rik...	default	crashpod	3	Deployment	crashpod	none -> ?	none -> none	none -> 10M	none -> 10M
2.	gke_robusta-development_us-east5-a_rik...	default	hamster	1	Deployment	hamster	100m -> 171m	300m -> none	50Mi -> 10M	none -> 10M
3.	gke_robusta-development_us-east5-a_rik...	default	ng.inx-deployment	3	Deployment	nginx	none -> 5m	none -> none	none -> 10M	none -> 10M
4.	gke_robusta-development_us-east5-a_rik...	default	nginx-deployment	3	Deployment	nginx	none -> 5m	none -> none	none -> 10M	none -> 10M
5.	gke_robusta-development_us-east5-a_rik...	kubewatch	ng.inx-deployment	1	Deployment	nginx	none -> 5m	none -> none	none -> 10M	none -> 10M
6.	gke_robusta-development_us-east5-a_rik...	robusta	inline-crashpod	1	Deployment	crashpod	none -> ?	none -> none	none -> 10M	none -> 10M
7.	gke_robusta-development_us-east5-a_rik...	robusta	robusta-forwarder	1	Deployment	kubewatch	10m -> 8m	none -> none	512Mi -> 37M	512Mi -> 37M
8. 9. 10.	gke_robusta-development_us-east5-a_rik...	robusta	robusta-grafana	1	Deployment	grafana-sc-dashboard grafana-sc-datasources grafana	none -> 9m none -> 5m none -> 5m	none -> none none -> none none -> none	none -> 97M none -> 93M none -> 79M	none -> 97M none -> 93M none -> 79M
11.	gke_robusta-development_us-east5-a_rik...	robusta	robusta-kube-prometheus-st-operator	1	Deployment	kube-prometheus-stack	100m -> 5m	none -> none	none -> 30M	none -> 30M
12.	gke_robusta-development_us-east5-a_rik...	robusta	robusta-kube-state-metrics	1	Deployment	kube-state-metrics	10m -> 5m	none -> none	none -> 19M	none -> 19M
13.	gke_robusta-development_us-east5-a_rik...	robusta	robusta-runner	1	Deployment	runner	250m -> 105m	none -> none	1Gi -> 918M	1Gi -> 918M
14. 15.	gke_robusta-development_us-east5-a_rik...	robusta	alertmanager-robusta-kube-prometheus-st-...	1	StatefulSet	alertmanager config-reloader	50m -> 5m 200m -> 5m	none -> none none -> none	200Mi -> 36M 50Mi -> 10M	none -> 36M 50Mi -> 10M
16. 17.	gke_robusta-development_us-east5-a_rik...	robusta	prometheus-robusta-kube-prometheus-st-pr...	1	StatefulSet	prometheus config-reloader	50m -> 201m 200m -> 5m	none -> none none -> none	none -> 1060M 50Mi -> 14M	none -> 1060M 50Mi -> 14M
18.	gke_robusta-development_us-east5-a_rik...	robusta	robusta-prometheus-node-exporter	3	DaemonSet	node-exporter	50m -> 5m	none -> none	none -> 16M	none -> 16M

<https://github.com/robusta-dev/krr>

One more thing {steve jobs}



Y la memoria de la JVM?

Usa límites del cgroup

-XX:+UseContainerSupport

Y la memoria de la JVM?

Evalúa forzar memoria física

-XX:+AlwaysPreTouch

Y la memoria de la JVM?

Haz que tu JVM muera ante la falta de memoria en heap

-XX:+ExitOnOutOfMemoryError

Ten Presente

Estabilidad sin rendimiento
es ineficaz.

Rendimiento sin estabilidad
es insostenible.

Ten Presente



**Kubernetes nos
da el equilibrio!**



`kubectl apply`



 /adalberto-garcia



 /adalj8d/kubernetes-resources-demo/